



Proyecto Intermodular: Planteamiento Inicial

Kajas: Gestión Inteligente de Almacenamiento

Alumno: David Lara Tarazaga
Grupo: 2ºDAW

25 de octubre de 2025

Índice

1. Elección del tema	3
1.1. Descripción de la idea o problema a resolver	3
1.2. Tipo de proyecto	3
1.3. Público objetivo o usuario final	3
1.4. Justificación de la elección	3
2. Análisis de necesidades	4
2.1. Identificación de las necesidades del usuario	4
2.2. Funcionalidades previstas	4
2.2.1. Funcionalidades principales (MVP)	4
2.2.2. Funcionalidades secundarias (Post-MVP)	4
2.3. Requisitos técnicos	5
2.4. Consideraciones sobre accesibilidad, usabilidad y diseño responsive	5
2.5. Posibles limitaciones o riesgos	5
3. Diseño de UI	6
4. Planificación del proyecto	7
4.1. Objetivos específicos y resultados esperados	7
4.2. Recursos materiales y humanos	7
4.3. Distribución de tareas	7
4.4. Cronograma general, Entregables e Hitos Parciales	7

1. Elección del tema

1.1. Descripción de la idea o problema a resolver

Kajas es una aplicación web (webapp) moderna y visual diseñada para la gestión de objetos, cajas y espacios de almacenamiento, tanto domésticos como profesionales.

El problema que resuelve es el caos y la incertidumbre generados por la acumulación de objetos. Muchas personas guardan pertenencias en cajas en trasteros, garajes o almacenes, y con el tiempo olvidan qué contienen exactamente o dónde está un objeto específico. Esto lleva a tener que abrir y rebuscar en múltiples cajas físicamente.

Kajas digitaliza este inventario mediante "cajas virtuales" vinculadas a códigos QR físicos. El usuario puede catalogar el contenido de una caja una sola vez y, posteriormente, simplemente escanear un código QR con su móvil para saber qué hay dentro sin necesidad de abrirla.

1.2. Tipo de proyecto

Se trata de una **aplicación web dinámica** (Web App) con un enfoque *mobile-first* (priorizando el móvil), dado que la funcionalidad clave de escaneo de QR se realiza con dicho dispositivo. Aunque será accesible vía web de escritorio para la gestión masiva, su principal utilidad se centra en la movilidad.

1.3. Público objetivo o usuario final

El público es amplio, pero se puede segmentar en:

- **Usuarios domésticos:** Familias con trasteros, personas que realizan mudanzas, coleccionistas o aficionados que necesitan inventariar su equipo.
- **Pequeños negocios (PYMES):** Profesionales con pequeño stock, oficinas que archivan documentos, o tiendas que manejan inventario en trastienda.
- **Usuarios de "self-storage":** Personas que alquilan espacios de almacenamiento externos y necesitan un control de lo que guardan.

1.4. Justificación de la elección

- **Aplicabilidad y Necesidad Real:** Resuelve un problema cotidiano, tangible y universal (la desorganización).
- **Viabilidad Técnica:** El stack tecnológico (React, Backend y BBDD) es estándar en la industria. La generación de QR es una tecnología madura y fácil de implementar.
- **Originalidad (Enfoque):** Aunque existen apps de inventario, muchas son complejas, caras (suscripciones) o tienen un diseño obsoleto. Kajas se diferenciará por su **identidad visual cozy** (amigable, minimalista y cálida) y su usabilidad centrada en la experiencia de usuario (UX).
- **Interés Académico:** El proyecto permite integrar tecnologías de Frontend (React), Backend (API REST), Base de Datos (gestión de relaciones) y una funcionalidad

específica (generación y uso de QR), cubriendo gran parte del currículo intermodular.

2. Análisis de necesidades

2.1. Identificación de las necesidades del usuario

El usuario necesita:

- Saber qué hay dentro de una caja sin abrirla.
- Encontrar un objeto específico ("¿Dónde guardé...?") rápidamente.
- Tener un inventario visual (con fotos) de sus pertenencias.
- Poder acceder a esta información desde su teléfono móvil en el lugar de almacenamiento (ej. trastero).
- Organizar sus pertenencias por ubicaciones (ej. "Garaje", "Armario 1").

2.2. Funcionalidades previstas

2.2.1. Funcionalidades principales (MVP)

- **Autenticación:** Registro e inicio de sesión de usuarios.
- **Gestión de Ubicaciones:** Crear, editar y eliminar espacios (ej. "Trastero", "Oficina").
- **Gestión de Cajas:** Crear una "caja virtual", asignarle nombre, descripción, foto y una ubicación.
- **Gestión de Objetos:** Añadir "objetos" dentro de una caja, con foto, nombre, descripción, cantidad, etc.
- **Generación de QR:** La app debe generar un código QR único para cada caja creada.
- **Escaneo de QR:** Al escanear el QR (usando la cámara del móvil vía web), la app debe abrir directamente la vista de esa caja y su contenido.
- **Buscador y Filtros:** Un buscador global para encontrar objetos o cajas por nombre, categoría o ubicación.

2.2.2. Funcionalidades secundarias (Post-MVP)

- **Panel de Estadísticas:** Un dashboard visual con resúmenes (Nº de cajas, Nº de objetos, peso estimado, valor).
- **Exportación:** Exportar inventario a Excel o JSON.
- **Modo Compartido:** Permitir que familiares o compañeros accedan al mismo inventario/ubicación.

2.3. Requisitos técnicos

Frontend: Vite + React.js. Para la interfaz se usará **Tailwind CSS** y componentes de **Shadcn/ui** para mantener una estética moderna y limpia.

Backend (Propuesta): Node.js con **Express** o **Fastify**. Creará una API RESTful para gestionar los datos.

Base de Datos (Propuesta): **PostgreSQL** (Relacional), ya que la estructura (Usuarios → Ubicaciones → Cajas → Objetos) es inherentemente relacional y se beneficiará de la integridad de datos.

Autenticación (Propuesta): **JWT** (JSON Web Tokens) para la gestión de sesiones de la API.

Almacenamiento (Propuesta): Un servicio de *storage* en la nube (como **AWS S3** o **Cloudinary**) para alojar las fotos de objetos y cajas.

Generación de QR: Se utilizará una biblioteca como **'qrcode.react'** en el frontend para la generación dinámica.

Servidor: Despliegue en plataformas como **Vercel** (Frontend) y **Render** o **Railway** (Backend + BBDD).

2.4. Consideraciones sobre accesibilidad, usabilidad y diseño responsive

- **Responsive (Mobile-First):** El diseño será *responsive* por definición. La funcionalidad de escaneo obliga a que la experiencia en el móvil sea fluida y nativa.
- **Usabilidad:** Interfaz limpia, minimalista y cálida (estilo *cozy*). Tipografía clara (Nunito/Poppins). Se evitará la sobrecarga de información, usando un diseño de "tarjetas".
- **Accesibilidad (A11y):** Se seguirán estándares básicos de WCAG: contraste de color adecuado (paleta suave pero legible), HTML semántico y atributos ARIA donde sea necesario.

2.5. Posibles limitaciones o riesgos

- **Coste de Almacenamiento:** El almacenamiento de fotos de usuario puede escalar en coste si la app crece mucho.
- **Gestión de QR Físicos:** El usuario debe imprimir la etiqueta QR. Si se daña, se pierde o se reasigna, puede crear confusión (Riesgo bajo, mitigable en la app).
- **Complejidad del "Modo Compartido":** La gestión de permisos y propiedad de los datos entre usuarios es compleja y se deja para una fase posterior al MVP.

3. Diseño de UI

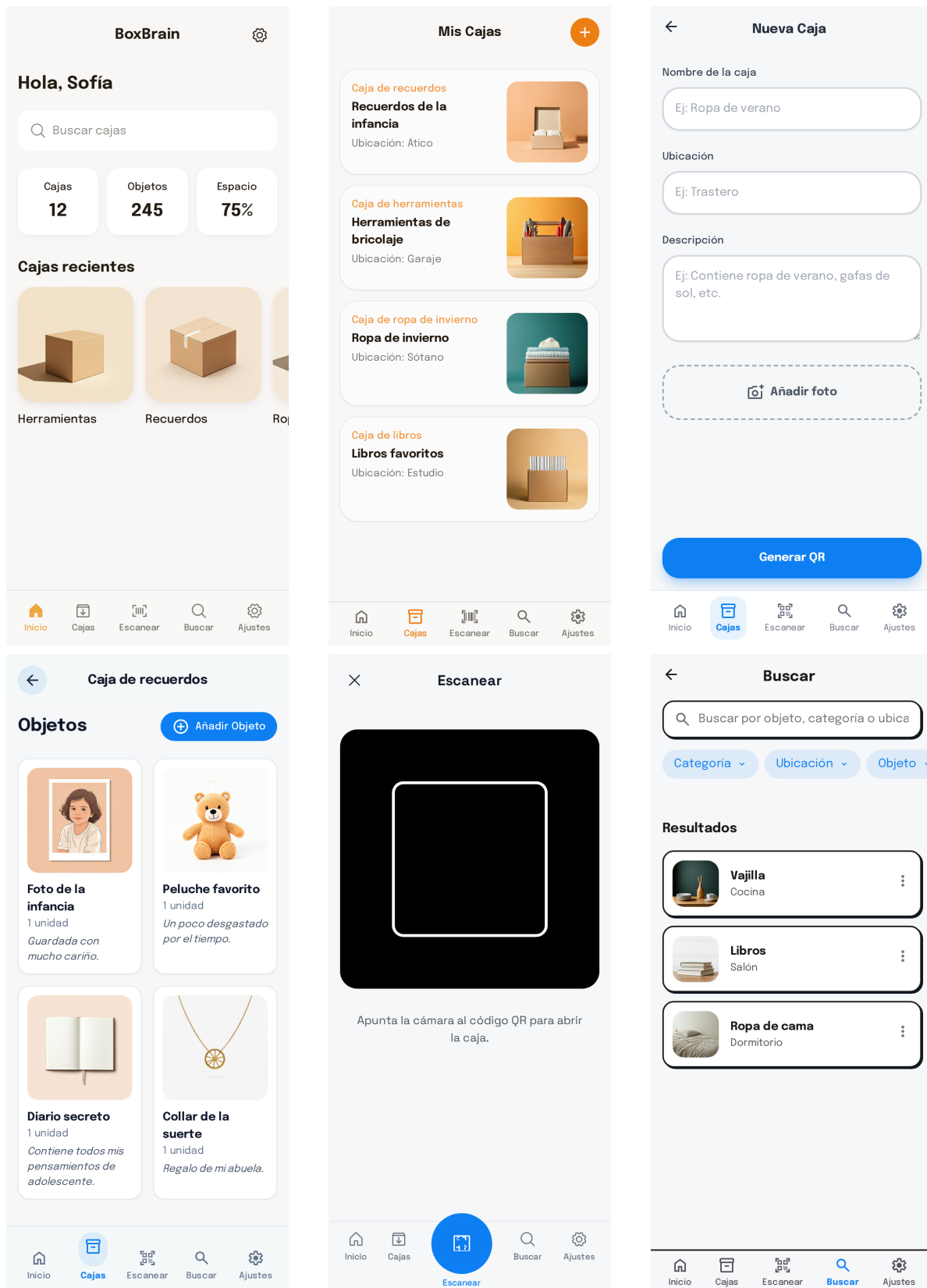


Figura 1: Ejemplos del diseño de la interfaz de usuario de Kajas.

4. Planificación del proyecto

4.1. Objetivos específicos y resultados esperados

El objetivo principal es desarrollar un Producto Mínimo Viable (MVP) funcional de Kajas antes de finalizar el curso, que cumpla con todas las funcionalidades principales descritas en la sección 2.2.1. El resultado esperado es una webapp desplegada y accesible públicamente.

4.2. Recursos materiales y humanos

- **Humanos:** 1 desarrollador (proyecto individual).
- **Materiales (Software):** VS Code, Git/GitHub, Figma (para bocetos), Trello (para cronograma).
- **Materiales (Hardware):** Ordenador personal y un teléfono móvil (para pruebas de escaneo QR).

4.3. Distribución de tareas

Al ser un proyecto individual, el alumno asumirá todos los roles: Project Manager, Diseñador UX/UI, Desarrollador Frontend y Desarrollador Backend.

4.4. Cronograma general, Entregables e Hitos Parciales

Se propone una planificación basada en hitos, gestionada visualmente en un tablero Trello o similar.

Hito 1: Análisis y Diseño (Semanas 1-3) ▪ *Entregable:* Documento de requisitos (este mismo), Wireframes de baja y alta fidelidad (Figma), y Diseño del Esquema de la Base de Datos.

Hito 2: Configuración y Backend (Semanas 4-7) ▪ *Entregable:* Repositorio Git configurado. API REST (Node.js) con endpoints para CRUD (Crear, Leer, Actualizar, Borrar) de Usuarios, Ubicaciones, Cajas y Objetos. Sistema de autenticación JWT funcional.

Hito 3: Desarrollo Frontend (Semanas 8-12) ▪ *Entregable:* Interfaz de usuario en React. Conexión completa con la API. Implementación de las vistas (Dashboard, vista de Caja, formulario de Objeto) y la lógica de generación de QR.

Hito 4: Funcionalidad Clave y Pruebas (Semanas 13-14) ▪ *Entregable:* Implementación de la función de escaneo de QR (con la cámara web). Pruebas E2E (End-to-End) del flujo completo: Crear caja → Imprimir QR → Escanear → Ver contenido.

Hito 5: Despliegue y Presentación (Semana 15) ▪ *Entregable:* Aplicación desplegada en la nube (Vercel/Render). Presentación final del proyecto.